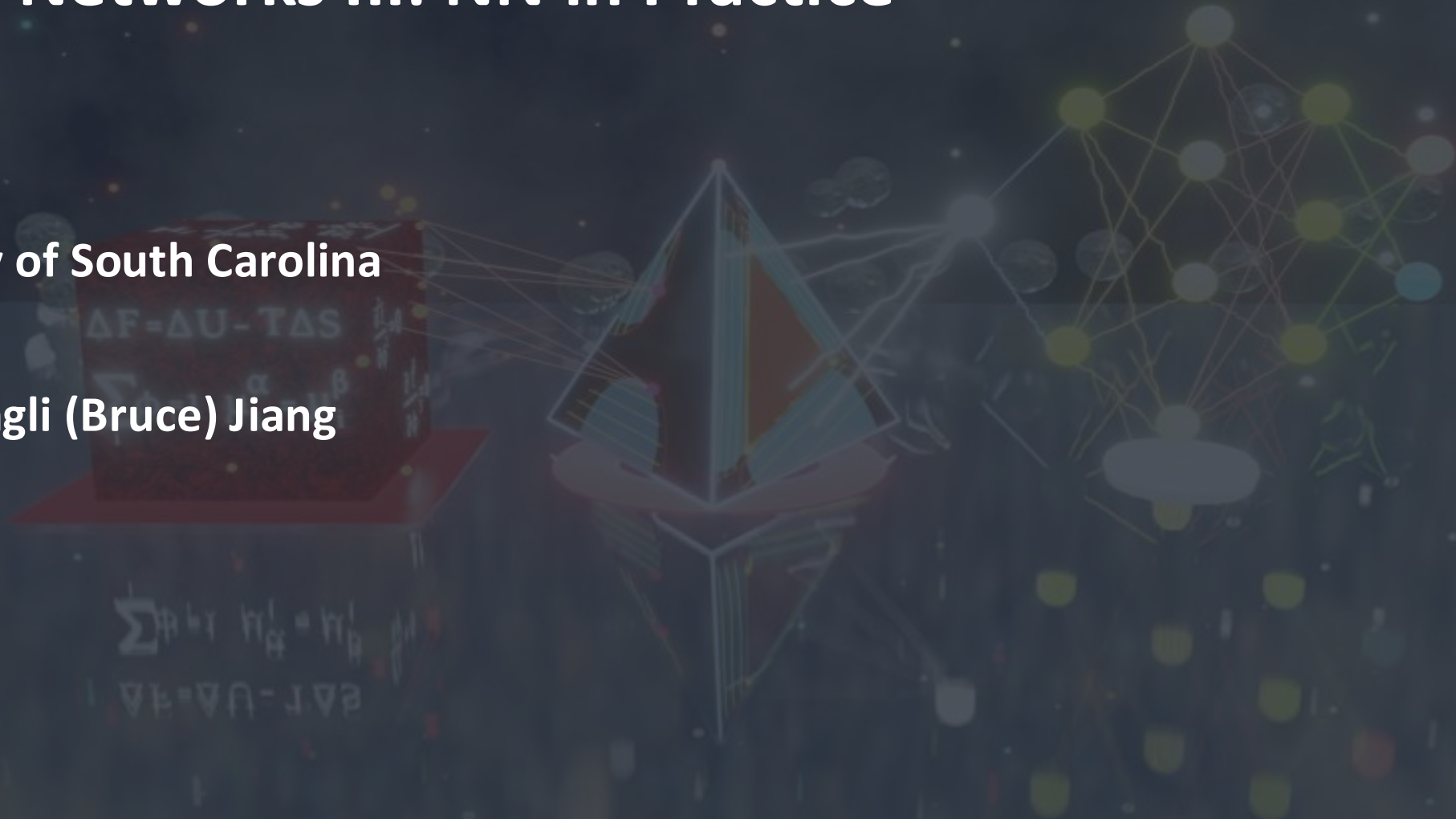


Neural Networks III: NN in Practice

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 18

Neural Networks II: Learning and Backpropagation

- *Loss functions, forward propagation, and backpropagation*
- *Gradient descent and updating weights and biases*
- *Train / validation / test splits and a first MNIST example*

In this lecture, you will learn

- *Epochs, batches, and learning rate*
- *Optimizers: SGD and Adam*
- *Training curves, overfitting, and regularization*

What happens inside model.fit()?

model.fit() repeatedly performs the training loop that we derived mathematically in Lecture 18.

For one mini-batch:

1. **Select a batch**
2. **Forward pass**
Inputs → predictions
3. **Calculate the batch loss**
4. **Backpropagation**
Calculate gradients of the loss
5. **Optimizer step**
Update weights and biases
6. **Take the next batch and repeat**

```
# 3. Hyperparameter tuning
models = {}

for n_hidden in [64, 128]:
    model = build_model(n_hidden)
    path = f"best_{n_hidden}.weights.h5"

    checkpoint = keras.callbacks.ModelCheckpoint(
        path, monitor="val_loss", save_best_only=True, save_weights_only=True)

    model.fit(x_train, y_train, validation_data=(x_val1, y_val1),
              epochs=20, batch_size=128, callbacks=[checkpoint], verbose=0)

    model.load_weights(path)
    models[n_hidden] = model

# Compare models using Validation 2
val_acc = {n: m.evaluate(x_val2, y_val2, verbose=0)[1]
            for n, m in models.items()}

best_hidden = max(val_acc, key=val_acc.get)
best_model = models[best_hidden]
```

What happens inside model.fit()?

model.fit() repeatedly performs the training loop that we derived mathematically in Lecture 18.

For one mini-batch:

1. **Select a batch**
2. **Forward pass**
Inputs $\rightarrow$ predictions
3. **Calculate the batch loss**
4. **Backpropagation**
Calculate gradients of the loss
5. **Optimizer step**
Update weights and biases
6. **Take the next batch and repeat**

After every training example has been used once, we say one epoch is “complete”.

$$\mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B \ell(y_i, \hat{y}_i)$$

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\text{batch}}$$

θ : all trainable weights and biases

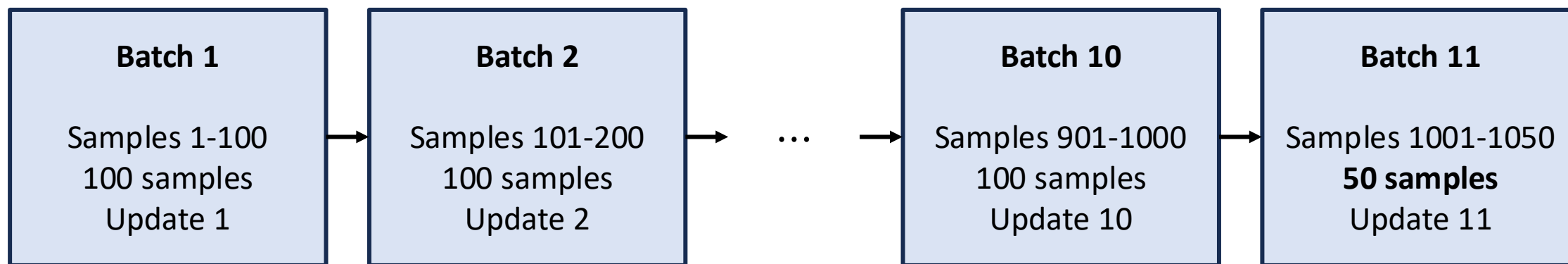
Are the weights updated once per epoch or once per batch?

Once per batch.

Epoch and Batch

We have 1,050 training examples and use a batch size of 100. The model processes one batch, updates its parameters, and then moves to the next batch.

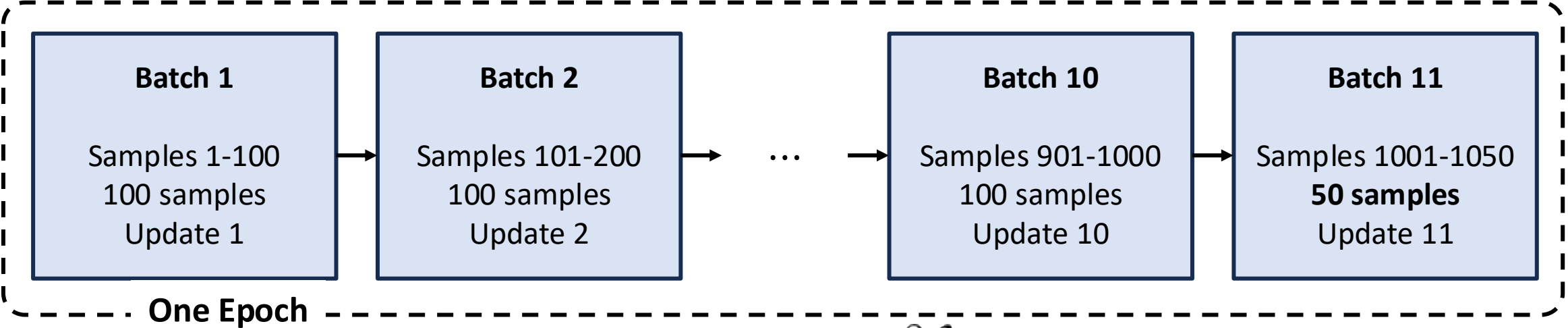
$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\text{batch}}$$



- Within a batch, all examples contribute to a loss calculated using the **same current parameters**.
- Backpropagation calculates the gradient of the **average batch loss**.
- The optimizer then updates **all trainable weights and biases**.
- The next batch uses these **updated parameters**.
- The final batch contains only 50 examples, but it still produces one update.

Epoch and Batch

Let's follow one weight, w . Start with a weight of 1 and use ordinary SGD with a learning rate of 0.1.



$$w_{\text{new}} = w_{\text{current}} - \eta \frac{\partial \mathcal{L}_{\text{batch}}}{\partial w}$$

Batch	Weights before this batch	Batch gradient	Calculation	Weight after update
1: samples 1-100	1	+2	$1 - 0.1 \times 2$	0.8
2: samples 101-200	0.80	+1	$0.8 - 0.1 \times 1$	0.7
3: samples 201-300	0.70	-0.5	$-0.7 - 0.1 \times (-0.5)$	0.75

Each batch starts from the latest parameter values, so learning accumulates across batches.

Why use batches?

Why not calculate the gradient using the entire training dataset every time?

Full batch



- Uses every training sample for one gradient.
- Only **one update per epoch**.
- Can be computationally expensive for large datasets.

Batch size = 1



- Many updates.
- Can make training less stable.
- May be computationally inefficient on modern hardware.

Why use batches?

Why not calculate the gradient using the entire training dataset every time?

Mini-batch



→ weight update

- Uses a subset of the data.

- Requires less memory.



→ weight update

- Gives many updates per epoch.

- Gradient contains some noise.



→ weight update

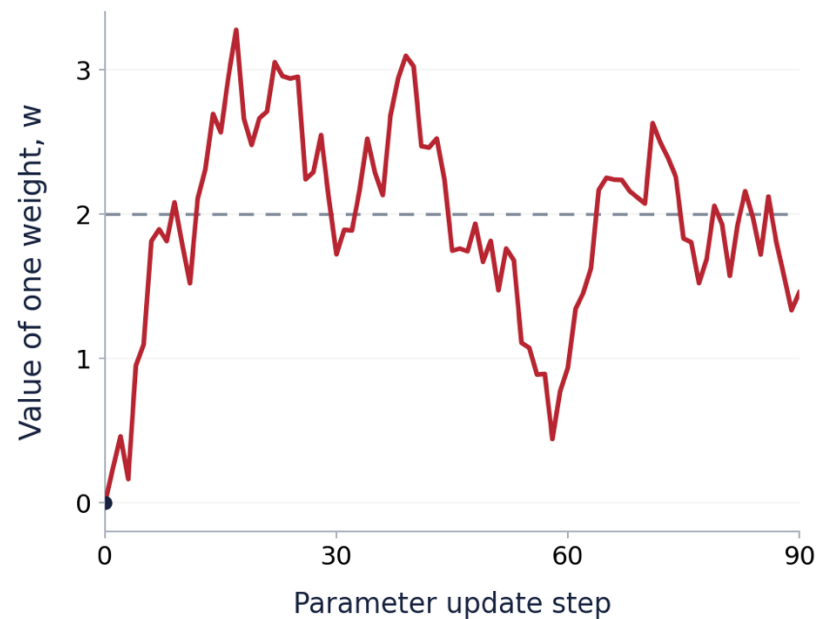
- Most neural networks are trained this way.

But there is no universally best batch size. Common starting values: 32, 64, 128, 256.

Batch size controls a trade-off between gradient stability, update frequency, memory use, and computation.

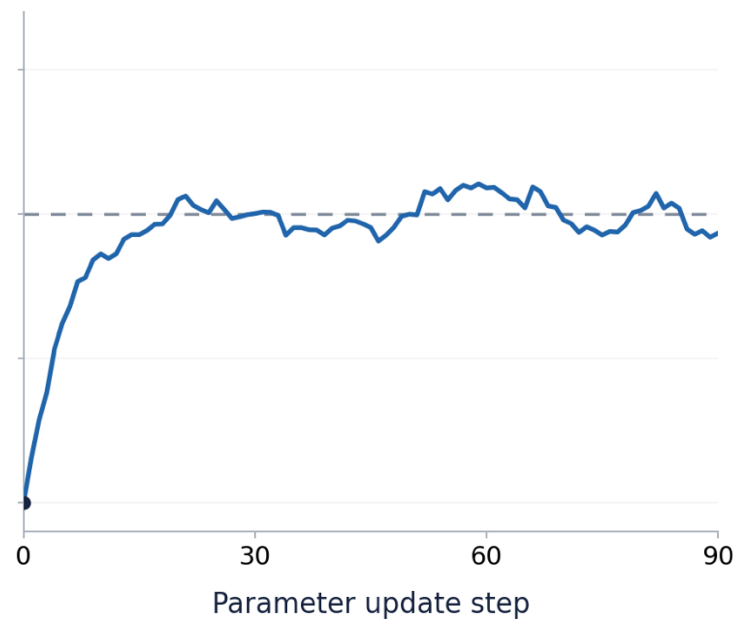
Why use batches?

Batch size = 1



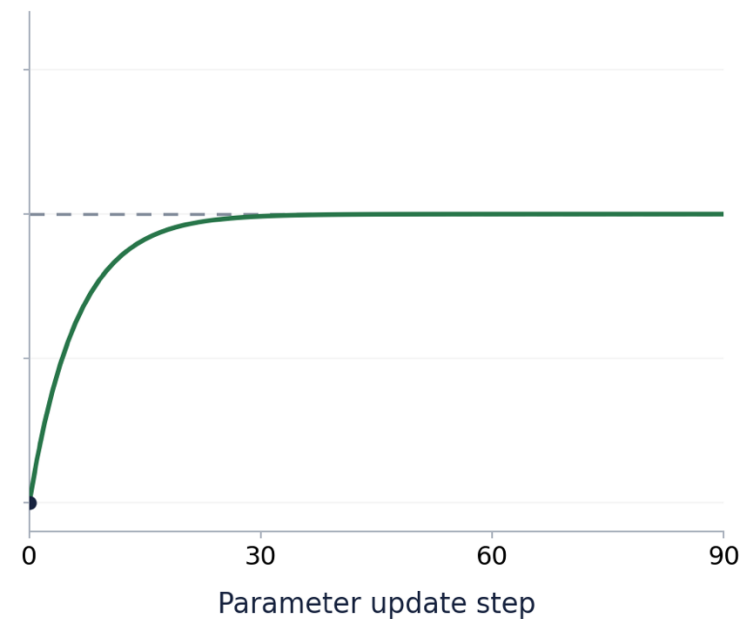
Large fluctuations

Mini-batch = 32



Smaller fluctuations

Full batch = 1050



Smooth path

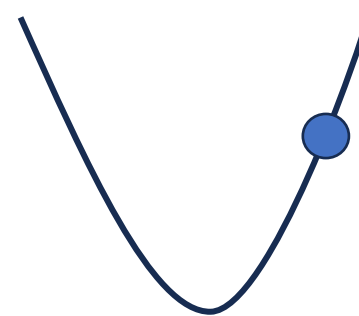
Learning Rate

The learning rate controls how far the optimizer moves after each gradient calculation.

$$\theta_{t+1} = \theta_t - \eta \mathbf{g}_t$$

$$\mathbf{g}_t = \nabla_{\theta} \mathcal{L}$$

- $\mathbf{g}_t$ tells us **which direction increases the loss**.
- Therefore, $-\mathbf{g}_t$ points toward decreasing loss.
- η , the learning rate, determines **how large the step is**.
- Small $\eta \rightarrow$ cautious updates.
- Large $\eta \rightarrow$ aggressive updates.



$$w = 2.0, \quad \frac{\partial \mathcal{L}}{\partial w} = 0.5$$

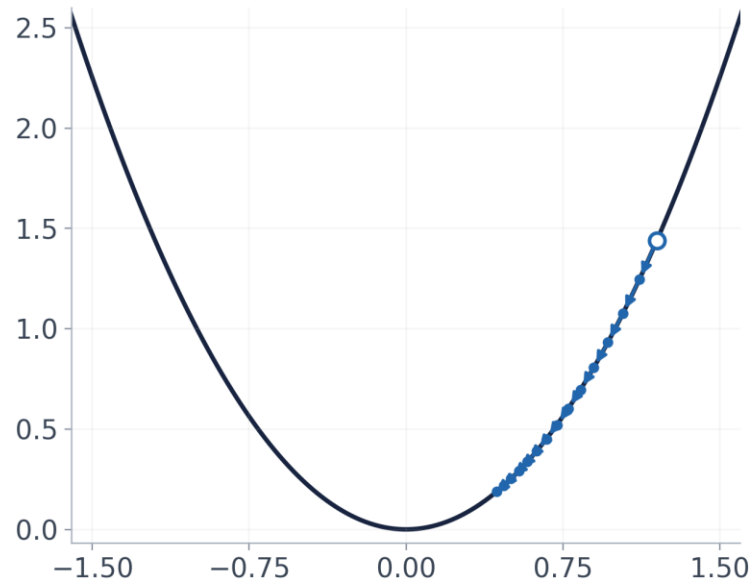
With $\eta = 0.1$, what is w_{new} ?

$$w_{\text{new}} = 2.0 - (0.1)(0.5) = 1.95$$

What if learning rate is...

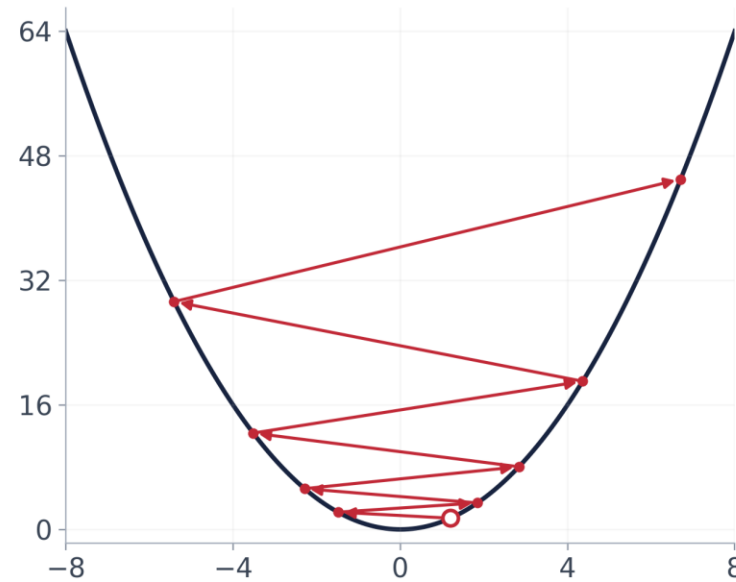
Too small

- Loss decreases, but very slowly.
- Many epochs may be required.
- Training can appear almost stuck.



Too large

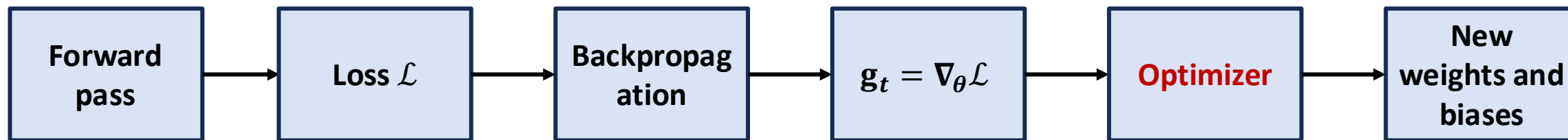
- Updates overshoot good parameter values.
- Loss may oscillate or increase
- Extremely large values can produce NaN.



But there is no universally best learning rate. Common starting values: 0.001, 0.005, 0.01.

Gradient Descent vs Optimizer

Backpropagation calculates gradients. The optimizer decides how those gradients are converted into parameter updates.



Gradient descent: basic principle

- Move parameters in the direction that reduces loss.
- Basic rule:

$$\theta_{t+1} = \theta_t - \eta g_t$$

Optimizer: specific update algorithm

Examples:

- **SGD**
- **RMSprop**
- **Adam**

Optimizers can use: 1) *the current gradient*, 2) *previous gradients*, 3) *gradient magnitude*, 4) *adaptive parameter-specific step sizes*.

Does Adam replace backpropagation? No. Backpropagation calculates the gradients; Adam uses those gradients.

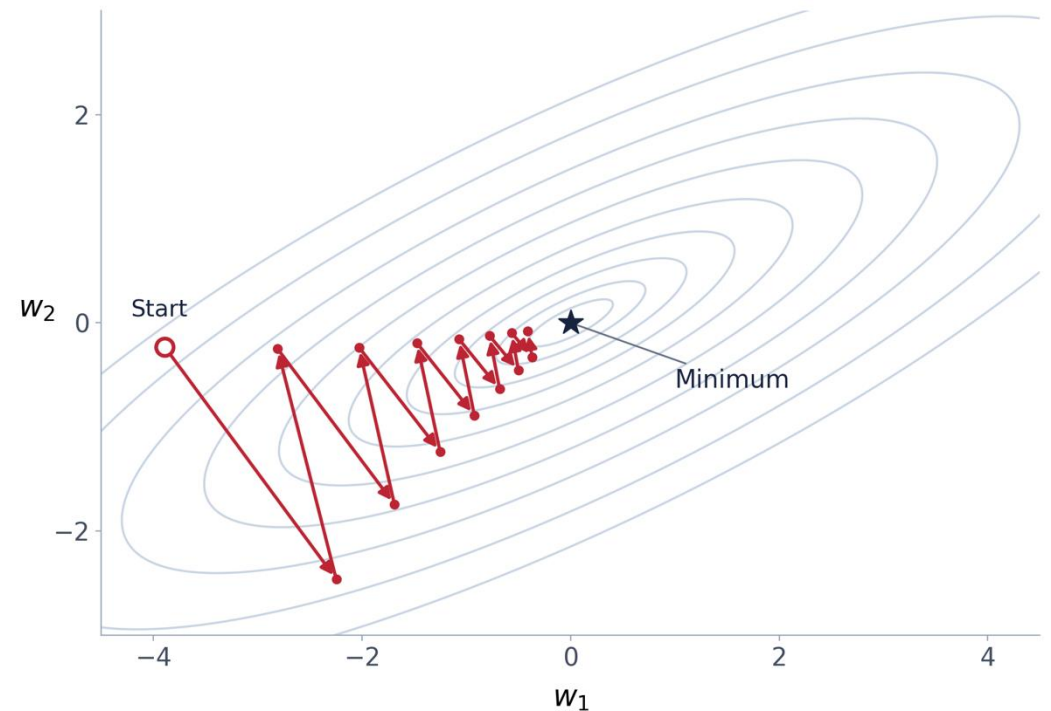
Stochastic Gradient Descent (SGD)

SGD applies the basic gradient-descent update to each mini-batch.

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{batch}}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{g}_t$$

- Uses the mini-batch gradient for each update.
- The same global learning rate is applied to all parameters.
- Mini-batch noise causes the update direction to vary from step to step.
- Performance can depend strongly on the learning rate.



Each arrow is a mini-batch update

Adam adapts the update for each parameter using information from previous gradients.

1. Remember the gradient direction

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2. Remember the gradient magnitude

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

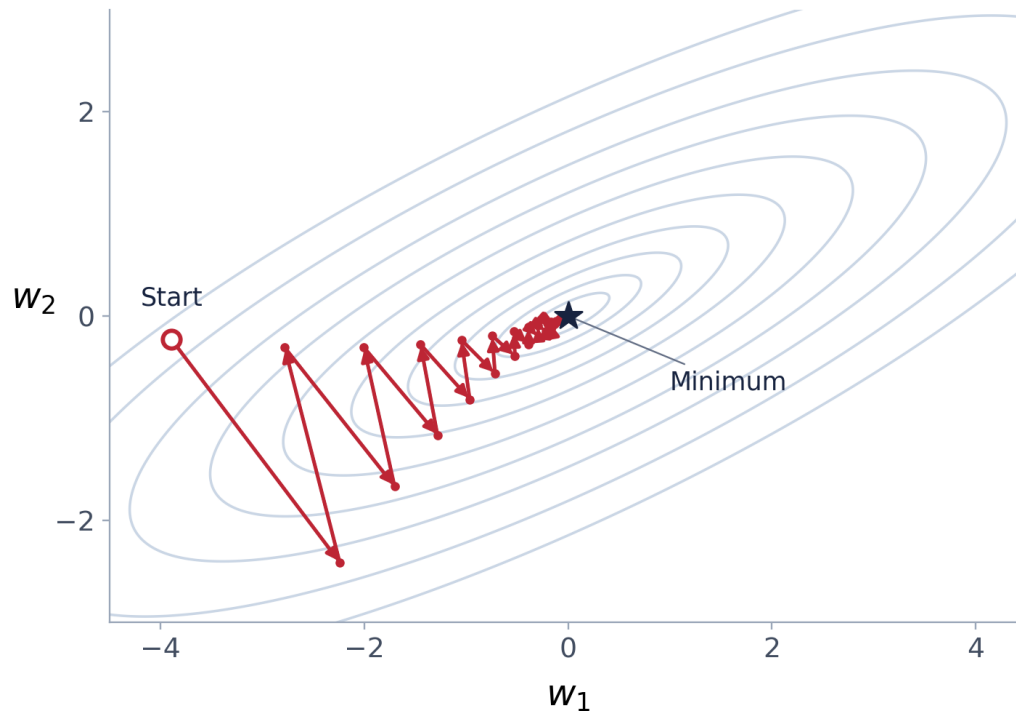
3. Use both to make the update

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

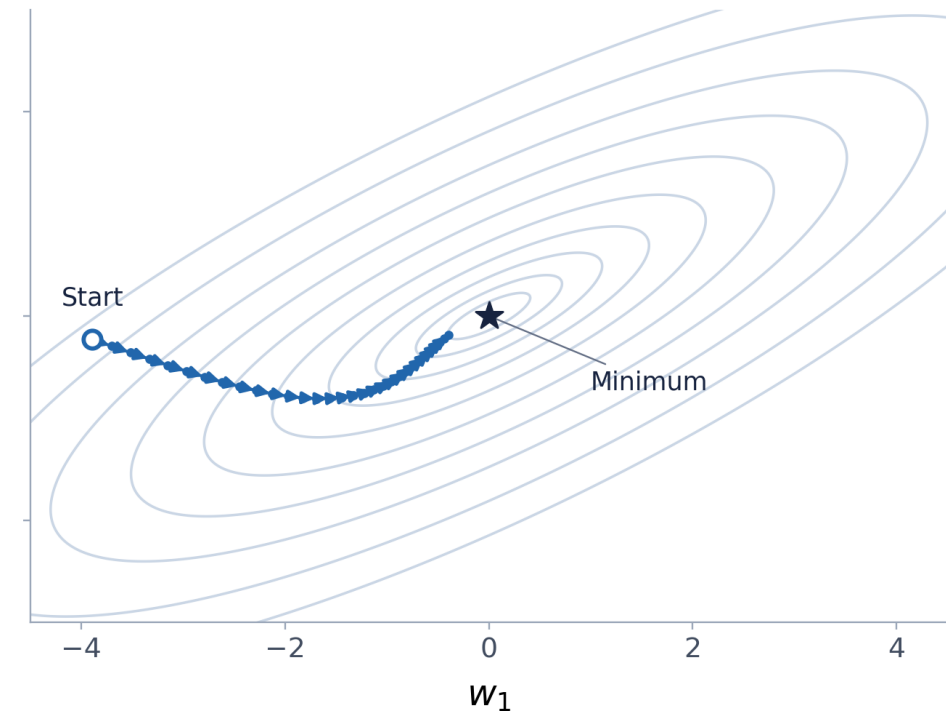
- m_t : moving average of gradients → **direction**
- v_t : moving average of squared gradients → **scale**
- Adam therefore gives different parameters different effective step sizes.
- It often works well without extensive tuning (β_1, β_2) are hyperparameters.
- **Adam still has a learning rate.**
- A common starting value is 1e-3.

Adam adapts the update for each parameter using information from previous gradients.

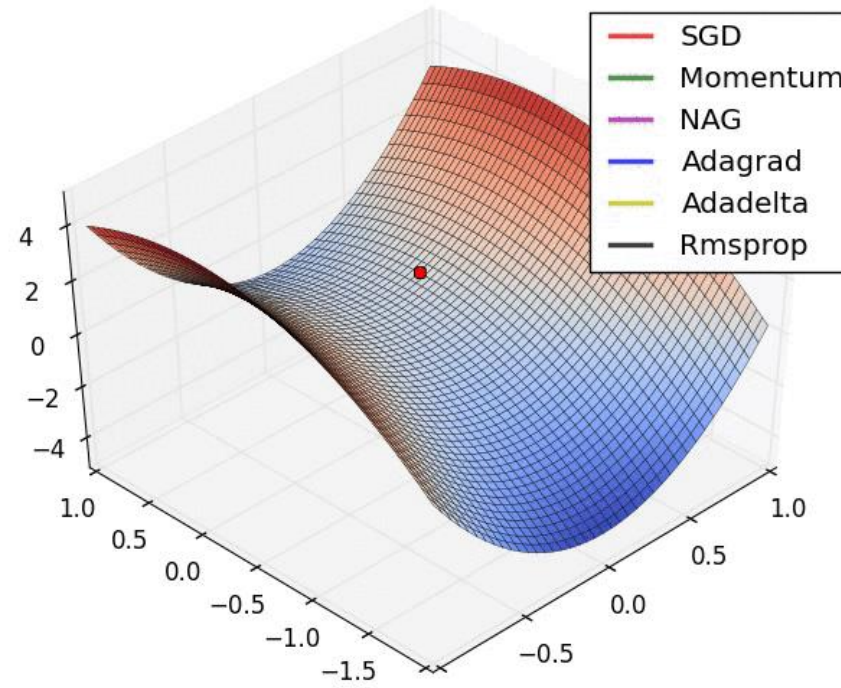
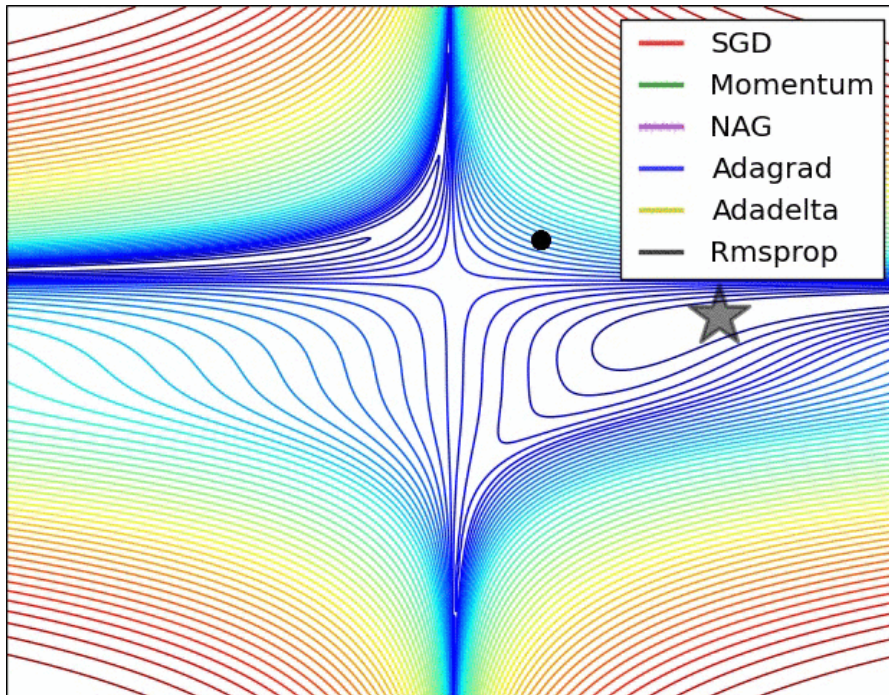
SGD



Adam



Different Optimizers



Images credit: [Alec Radford](#).

Training Curves

Training curves show how model performance changes as learning proceeds.

After every epoch, record:

- **Training loss**
- **Validation loss**
- Training accuracy
- Validation accuracy

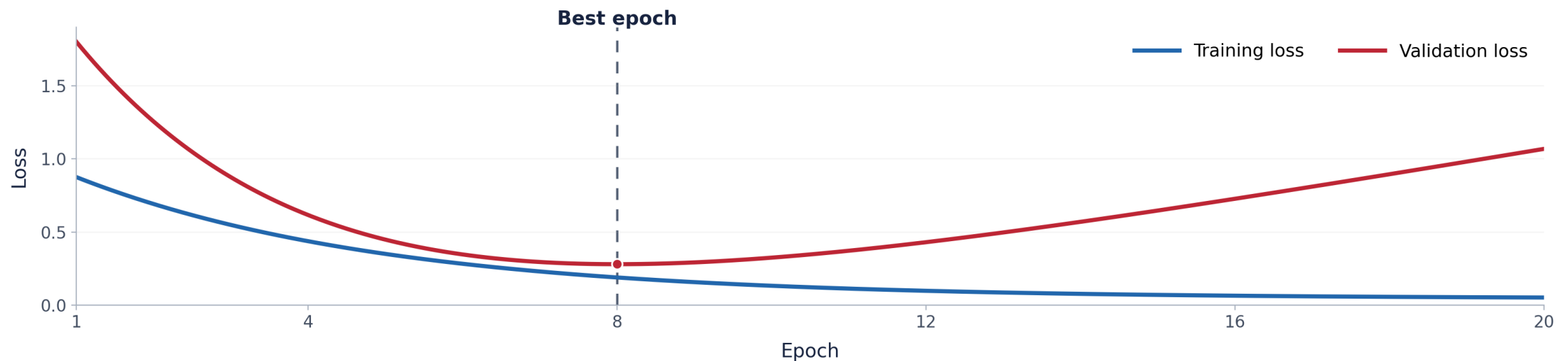
Training curve:

How well are we fitting examples used for learning?

Validation curve:

How well are we generalizing to unseen examples?

For diagnosing training, loss is often more informative than accuracy.



Underfitting

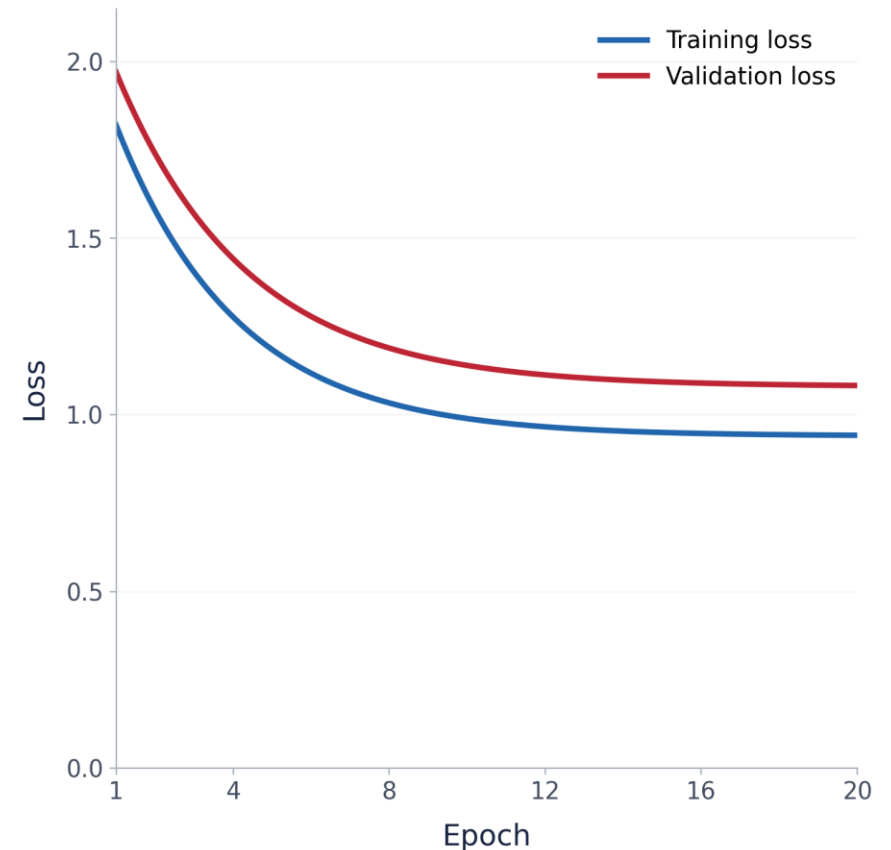
Underfitting occurs when the model has not learned the training data well enough.

Symptoms

- Training loss is still relatively high.
- Validation loss is also high.
- Training and validation curves are often close together.
- Training accuracy itself is unsatisfactory.

Possible causes

- Model has too little capacity.
- Training stopped too early.
- Learning rate is too small.
- Input representation does not contain enough useful information.



Overfitting

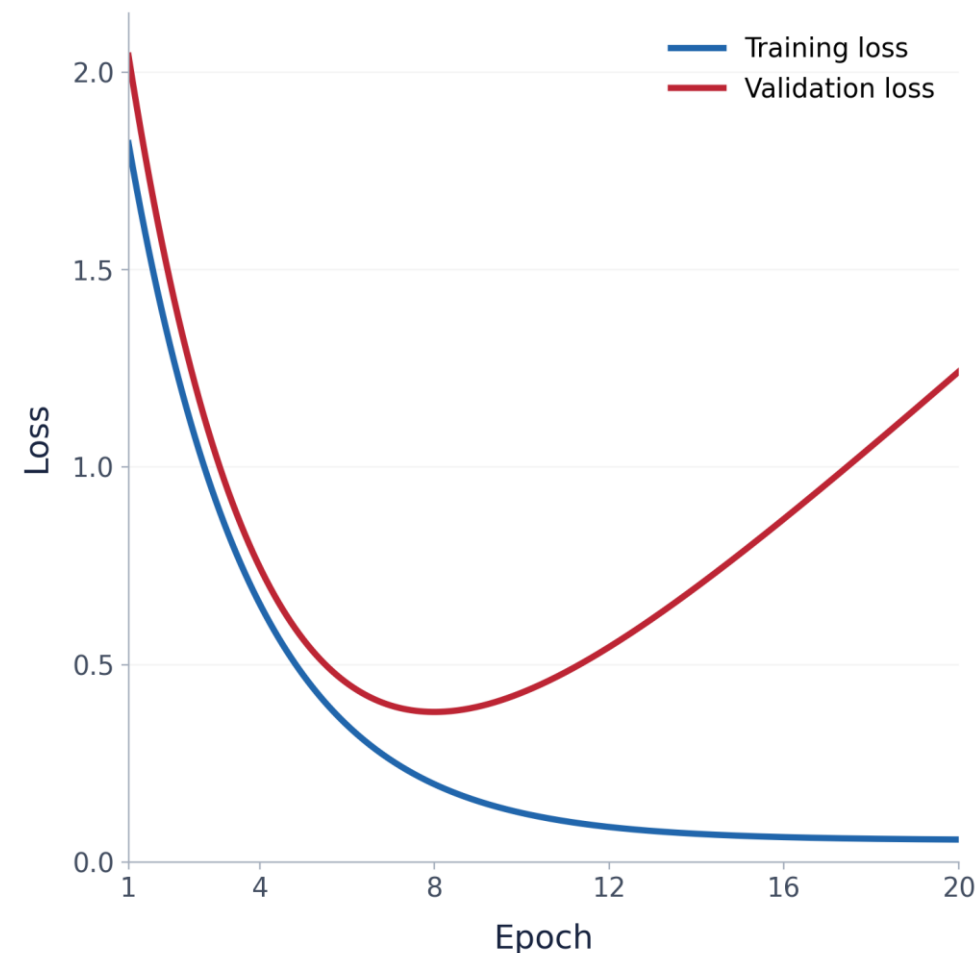
Overfitting occurs when training performance keeps improving while validation performance stops improving.

Symptoms

- Training loss continues decreasing.
- Validation loss reaches a minimum and then increases.
- A growing gap appears between training and validation performance.

Possible causes

- The model begins fitting:
 - training-specific details,
 - noise,
 - or patterns that do not generalize.



Regularization

Regularization reduces overfitting by limiting how strongly the model can adapt to the training data.

Early stopping

- Monitor validation loss.
- Stop when validation performance stops improving.
- Keeps the weights from the best epoch.

```
early_stop = tf.keras.callbacks.EarlyStopping(patience=10)
model.fit(X_train, y_train, validation_data=(X_val,
y_val), epochs=100, callbacks=[early_stop])
```

L2 regularization

- Penalizes large weights.
- Encourages smoother, less extreme models.

```
tf.keras.layers.Dense(
    128,
    activation="relu",
    kernel_regularizer=tf.keras.regularizers.l2(0.001))
```

Dropout

- Randomly removes some hidden activations during training.
- Prevents neurons from relying too strongly on one another.

```
tf.keras.layers.Dropout(0.2)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation="softmax")])
```

Control model capacity

- Fewer neurons/layers reduce the number of trainable parameters.
- A simpler model can generalize better when data are limited.

Quiz

A neural network has 48,000 training examples and uses `batch_size = 128`.

Approximately how many parameter updates occur during one epoch?

Answer: 375

Training loss keeps decreasing, but validation loss reaches a minimum at epoch 8 and then begins increasing. What is the best interpretation?

- A. The network needs more classes
- B. The model begins overfitting after about epoch 8
- C. The learning rate must be exactly zero
- D. The training data are not being used

Answer: B. The model begins overfitting after about epoch 8.

Interactive Python Time

`19_NN_in_practice.ipynb`

In this lecture, we learned

- *Epochs, batches, and learning rate*
- *Optimizers: SGD and Adam*
- *Training curves, overfitting, and regularization*

Next lecture:

- *Unsupervised Learning*
 - *Learning patterns from data without target labels*
 - *Dimensionality reduction with PCA*
 - *Introduction to clustering and other unsupervised methods*